

CS464 Final Project Report

Deep Learning Image Colorization

Group 20

Ozan Sazak - 22002997
Yılmaz Öncü - 22103780
Mustafa Boran Kaya - 22102900
Emre Oral - 22003050
Umut Yalçındağ - 22101831

1. Introduction

A classic, ill-posed problem in computer vision is image colorization, which is the act of reconstructing a believable RGB image from a single luminance (grayscale) input. A single gray pixel with an intensity of 0.5 may conceptually correlate to the red of a brick structure, the green of a forest canopy, or the gray of asphalt. This is why it is referred to as "ill-posed" since the mapping from grayscale to color is multimodal. There are just reasonable answers; there isn't a single "correct" response.

In this project, we explored the limitations of standard regression-based approaches to this problem. Traditional Convolutional Neural Networks (CNNs) trained with Mean Squared Error (MSE) loss functions typically fail to capture this multimodality. Since MSE seeks to minimize the average error across all possible pixel values, the model tends to predict the statistical mean of the color distribution. Visually, this results in the "**Sepia Effect**": outputs are being desaturated, brownish, and lacking in vibrancy.

Our research objective was to develop a system that overcomes this desaturation problem by explicitly conditioning the model on global color statistics. We iterated through three distinct architectural phases: a Baseline CNN Autoencoder, a Standard U-Net to recover spatial resolution, and finally, a **Histogram-Guided Fusion U-Net**. This final architecture treats colorization not just as a reconstruction task, but as a fusion task, merging local spatial features with global color priors to produce vibrant, user-controllable results.

2. Dataset and Preprocessing Pipeline

We made use of the Kaggle-sourced Intel **Image Classification Dataset**. We used this dataset for a self-supervised image-to-image regression challenge, even though it was initially selected for scene categorization tasks.

2.1 Dataset Characteristics

The dataset is divided into six different classes: Buildings, Forest, Glacier, Mountain, Sea,

and Street which are almost equally distributed in our dataset. It includes about 14,000 training images and 3,000 test images. For our particular strategy, this class variety is essential. We can test if our Fusion model can effectively learn to apply particular color palettes to suitable semantic structures since each class has a unique color distribution (for example, Sea is blue-dominant, Forest is green-dominant).

2.2 The Preprocessing Pipeline

To ensure compatibility with deep autoencoder architectures and optimal convergence, we implemented a rigorous preprocessing pipeline.

1. Converting Dataset to the Grayscale:



Fig 1: Building Example (Real / Converted)



Fig 2: Sea Example (Real / Converted)



Fig 3: Forest Example (Real / Converted)



Fig 4: Glacier Example (Real / Converted)



Fig 5: Mountain Example (Real / Converted)



Fig 6: Street Example (Real / Converted)

2. Geometric Transformation (Resizing):

A critical engineering decision was the resizing of all input images from their original dimensions to 160x160 pixels.

Standard U-Net architectures utilize a series of down-sampling operations (typically Max Pooling) to compress spatial information. Our model employs a depth of 4, meaning the input is divided by 2 four times ($2^4 = 16$).

- If we used a standard size like 150x150, the dimensions would become non-integers at the bottleneck (150 to 75 to 37.5), forcing the framework to apply padding strategies that often introduce artifacts during the up-sampling (decoding) phase.
- By enforcing a resolution of **160x160**, we ensure clean divisibility (160 to 80 to 40 to 20), resulting in a perfectly aligned bottleneck tensor of shape (20, 20, 512).

3. Normalization:

Pixel intensities were cast to float32 and scaled from the range [0, 255] to [0, 1]. This normalization is essential for stabilizing gradients and ensuring the input range matches the output range of our final Sigmoid activation layer.

4. Color Space Conversion:

For every sample, we generated two tensors:

- **Input (X)**: The image converted to Grayscale (160, 160, 1) .
- **Target (Y)**: The original RGB image (160, 160, 3).

5. Histogram Extraction (The "Prior"):

Unlike standard pipelines, our preprocessing included a feature extraction step for the fusion layer. For every image in the training set, we computed a Color Histogram.

- We utilized 32 bins for each of the 3 RGB channels.
- These distributions were concatenated into a single **96-dimensional vector**.
- This vector successfully resolves the multimodal ambiguity mentioned in the introduction by acting as the "hint" or "prior" that informs the model about the colors present in the scene.

6. Stratified Splitting:

We used a stratified shuffle split to avoid class imbalance distorting our validation measures. In order to maintain the same distribution of the six classes (Forest, Sea, etc.) throughout the training and validation sets, we set aside 20% of the data for validation.

3. Methodology and Architecture

Our research methodology followed an iterative trajectory. We identified that image colorization is not merely a reconstruction task but a generative one requiring semantic guidance. Consequently, we moved from a naive regression model to a structurally complex, multi-modal network.

3.1 Architectural Evolution: From CNN to U-Net

Phase 1: The Baseline Autoencoder

Our initial experiments utilized a standard Convolutional Encoder-Decoder. This network compressed the input image into a latent representation and attempted to upscale it back to RGB dimensions.

Technical Specifications of the Baseline Model

The Baseline Autoencoder features a shallow, symmetrical architecture designed to establish a performance floor. It consists of three down-sampling blocks and three up-sampling blocks with no residual connections.

- **Depth**: 6 Convolutional Layers (excluding input/output).
- **Filter Progression**: 64 → 128 → 256 (Bottleneck) → 128 → 64.
- **Parameter Count**: Approximately 1.2 Million.

Table 1: Baseline CNN Architecture Details

Stage	Layer Type	Filter Size	Activation	Output Shape	Description
Input	Input Layer	-	-	(160, 160, 1)	Grayscale Input
Encoder 1	Conv2D + BN	64	ReLU	(160, 160, 64)	Initial Feature Extraction

	MaxPooling2D	-	-	(80, 80, 64)	Spatial Compression
Encoder 2	Conv2D + BN	128	ReLU	(80, 80, 128)	Intermediate Features
	MaxPooling2D	-	-	(40, 40, 128)	Spatial Compression
Bottleneck	Conv2D + BN	256	ReLU	(40, 40, 256)	Latent Representation
	MaxPooling2D	-	-	(20, 20, 256)	Maximum Compression
Decoder 1	Conv2D + BN	256	ReLU	(20, 20, 256)	Reconstruction
	UpSampling2D	-	-	(40, 40, 256)	Spatial Expansion
Decoder 2	Conv2D + BN	128	ReLU	(40, 40, 128)	Feature Refinement
	UpSampling2D	-	-	(80, 80, 128)	Spatial Expansion
Decoder 3	Conv2D + BN	64	ReLU	(80, 80, 64)	High-level Details
	UpSampling2D	-	-	(160, 160, 64)	Return to Original Size
Output	Conv2D	3	Sigmoid	(160, 160, 3)	RGB Prediction

- **The Failure Mode:** The outputs suffered from severe spatial blurring and color averaging.
- **Technical Diagnosis:** As the image passes through pooling layers, spatial resolution is lost.

3.2 The Proposed Solution: Histogram-Guided Fusion U-Net

Our final model addresses the limitations of the baseline by integrating global color priors with spatial feature maps.

Result: The Decoder receives a fused tensor of shape (20, 20, 768). This forces the network to reconstruct the image using the spatial structure of the original input conditioned on the color constraints provided by the histogram.

Technical Specifications of the Fusion U-Net

Our proposed model is significantly deeper and more complex than the baseline. It employs a "Double Convolution" block structure (two consecutive Conv2D-BN-ReLU operations per level) to increase non-linearity and feature retention. The architecture integrates the histogram fusion mechanism at the deepest bottleneck layer.

- **Structure:** 4 Encoder Levels + Fusion Bottleneck + 4 Decoder Levels.
- **Filter Progression:** 64 → 128 → 256 → 512 (Bottleneck).
- **Fusion Depth:** The bottleneck handles a concatenated tensor depth of **768 channels** (512 image features + 256 histogram features).
- **Parameter Count:** Approximately 8.5 Million (roughly 7x larger than the Baseline).

Table 2: Histogram-Guided Fusion U-Net Architecture

Stage	Operation	Tensor Shape	Function
Input (Image)	Input Layer	(160, 160, 1)	Grayscale Spatial Input
Input (Hist)	Input Layer	(96,)	Color Probability Distribution
Encoder L1	Double Conv Block	(160, 160, 64)	Low-level features (Edges/Textures)
	MaxPooling	(80, 80, 64)	Downsampling
Encoder L2	Double Conv Block	(80, 80, 128)	Mid-level features
	MaxPooling	(40, 40, 128)	Downsampling
Encoder L3	Double Conv Block	(40, 40, 256)	High-level semantic features
	MaxPooling	(20, 20, 256)	Downsampling
Encoder L4	Double Conv Block	(20, 20, 512)	Deepest Spatial Features
Histogram MLP	Dense -> Reshape -> Tile	(20, 20, 256)	Spatially expanded color priors
FUSION	Concatenate	(20, 20, 768)	Merging Content + Style
Bottleneck Conv	Conv2D (1x1)	(20, 20, 512)	Feature Integration
Decoder L1	UpSample + Skip(L3)	(40, 40, 768)	Recovering L3 spatial details
	Double Conv Block	(40, 40, 256)	Channel reduction
Decoder L2	UpSample + Skip(L2)	(80, 80, 384)	Recovering L2 spatial details
	Double Conv Block	(80, 80, 128)	Channel reduction
Decoder L3	UpSample + Skip(L1)	(160, 160, 192)	Recovering L1 spatial details
	Double Conv Block	(160, 160, 64)	Final feature map
Output	Conv2D (Sigmoid)	(160, 160, 3)	Final Colorization

3.3 Loss Function Optimization: MAE vs. MSE

A critical finding of our study was the impact of the loss function on color vibrancy.

- **Mean Squared Error (MSE):** Large mistakes are disproportionately penalized by MSE. guessing "gray" in an ambiguous situation (such as an automobile that may be red or blue) has a smaller mean square error (MSE) than guessing "red" if the ground truth is "blue." This led to the "Sepia Effect."
- **Mean Absolute Error (MAE):** We switched to MAE for the final model. MAE is less sensitive to outliers and encourages the model to predict the *median* of the conditional distribution rather than the *mean*. This allows the model to make "bolder"

color predictions, significantly improving saturation.

3.4 Post-Processing: Saturation Boosting

Despite the architectural improvements, deep regression models can still be conservative. To counteract this, we implemented a deterministic post-processing step:

1. The RGB output is converted to the **HSV (Hue, Saturation, Value)** color space.
2. The **Saturation (S)** channel is multiplied by a factor of $\gamma = 1.4$ (a 40% boost).
3. Values are clipped to the range $[0, 1]$ to prevent artifacts.
4. The image is converted back to RGB.

This final polish ensures that the visual output matches the vibrancy of natural scenes, particularly for classes like *Forest* and *Sea*.

4. Experimental Setup and Optimization

Training high-dimensional generative models requires substantial computational throughput. To address the heavy matrix multiplication loads inherent in our 160x160 U-Net architecture, we engineered an optimized training pipeline.

4.1 Hardware and Environment

All experiments were conducted using the **Google Colab Pro** cloud infrastructure. We utilized a single **NVIDIA A100 GPU**, a high-end accelerator capable of handling the substantial VRAM requirements of our fusion bottleneck layer.

4.2 Computational Optimization: Mixed Precision

We activated TensorFlow's Mixed Precision training strategy (`mixed_float16`) in order to optimize the A100 architecture's efficiency.

- **Mechanism:** For numerical stability, this method maintains master weights in single-precision (FP32) while executing computationally demanding operations (convolutions, matrix multiplications) at half-precision (FP16).
- **Impact:** We were able to expand the batch size to 64 without experiencing Out-Of-Memory (OOM) issues because of this modification, which practically quadrupled our training pace and drastically decreased VRAM utilization.

4.3 Hyperparameter Configuration

The Adam optimizer was used to train the model because of its flexible learning rate capabilities, which are ideal for non-convex optimization issues such as image production.

4.4 Regularization Strategy: Early Stopping

Given that our model receives explicit color "answers" via the histogram input, there is a theoretical risk of the network memorizing the mapping between specific histogram vectors and training images rather than learning the generalized semantic rules.

To mitigate this, we implemented an **Early Stopping** mechanism.

- **Configuration:** We set the maximum training duration to **100 epochs**.
- **Patience:** With ten epochs of patience, the training loop tracked the Validation Loss. Training was automatically stopped and the weights from the best-performing epoch were restored if the model did not increase its performance on unseen data for ten consecutive epochs. By doing this, we were able to capture the model at its best for generalization.

5. Results and Discussion

In this section, we present a comprehensive evaluation of our three architectural phases: the Baseline CNN, the Class-Average Fusion U-Net (Method A), and the Reference-Based Fusion U-Net (Method B). We analyze the performance of each model through both quantitative metrics (loss convergence profiles) and qualitative assessments (visual inspection of the generated images), highlighting the progression from simple regression to controllable generation.

5.1 Baseline CNN Autoencoder Evaluation

5.1.1. CNN Training and Validation Loss Analysis In the initial phase of our research, we trained a simple Convolutional Encoder-Decoder using the Mean Squared Error (MSE) loss function. The training profile for this 30-epoch run reveals a distinct optimization behavior. As illustrated in the loss chart, the model achieved a rapid descent in error during the first five epochs, quickly plateauing at a very low numerical value, with a final validation loss of approximately 0.0073.

However, this low numerical error is deceptive. MSE minimizes the squared difference between pixel values, which mathematically encourages the model to predict the statistical mean of all plausible colors when faced with uncertainty. While the training curve suggests a stable and "successful" convergence, the model was essentially minimizing its penalty by avoiding bold color choices, settling instead for a safe, average prediction.

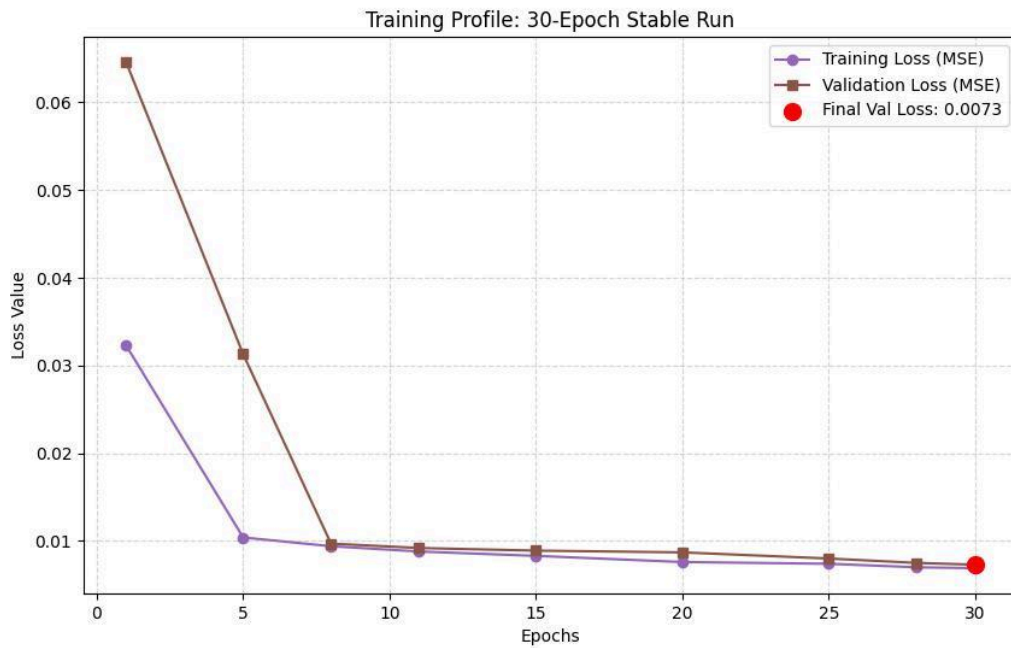


Fig 7: Training and Validation Loss Scores (MSE) of CNN - 30 epoch

5.1.2. CNN Visual Evaluation: The Sepia Effect When examining the visual outputs of the Baseline CNN, the limitations of the MSE loss function become immediately apparent. The resulting images suffer heavily from what is known in the literature as the "Sepia Effect". Because the "average" of the RGB color spectrum often resolves to brown or gray hues, the model's predictions appear washed out and monochromatic.

Furthermore, the lack of structural retention is evident. Without the skip connections found in U-Net architectures, the encoder-decoder structure failed to preserve spatial information during down-sampling, resulting in outputs that were not only desaturated but also significantly blurry. The visual evidence confirms that a standard autoencoder is insufficient for recovering high-frequency details or generating vibrant color.

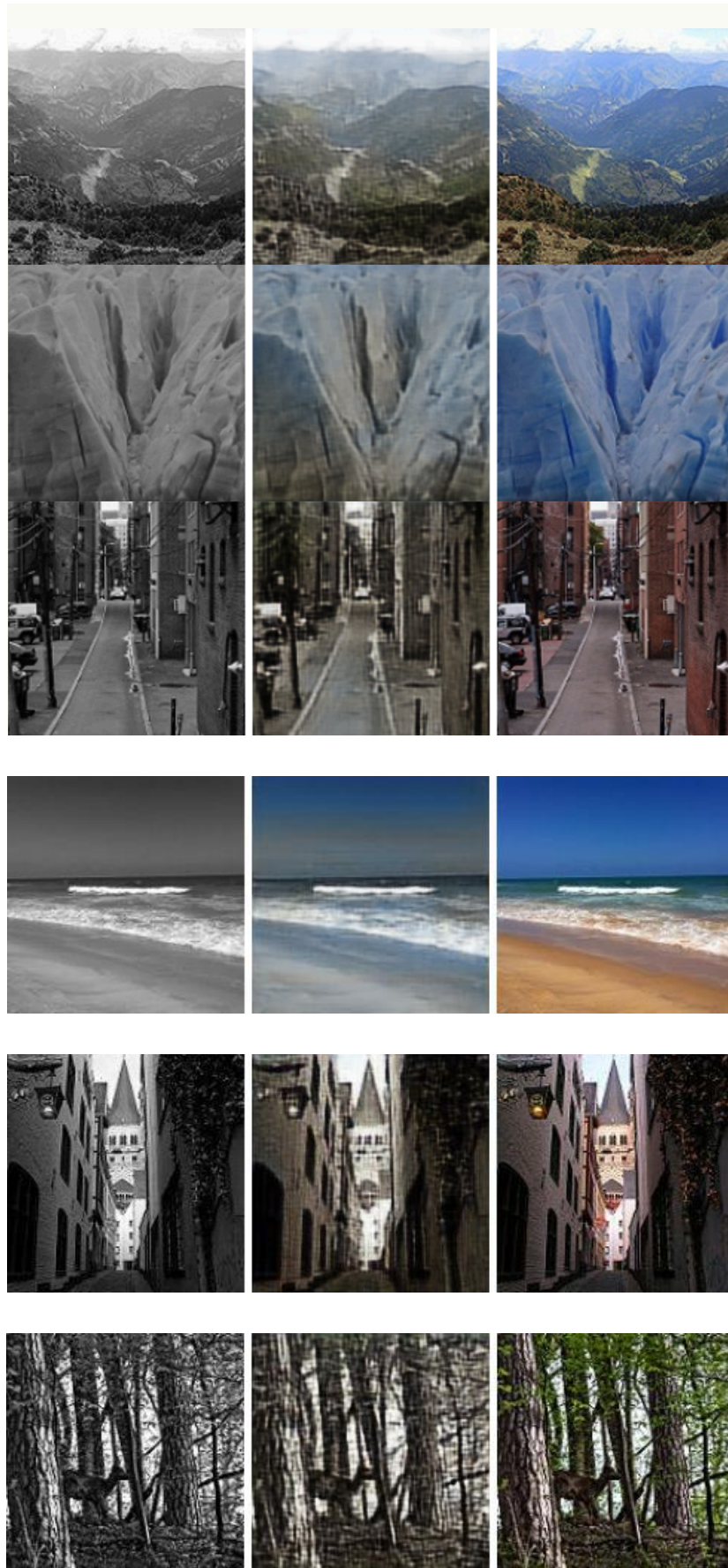


Fig 8: CNN output: Grayscale / Colorized by Standard CNN / Real image (Mountain, Glacier,

Street, Sea, Buildings, Forest consequently)

Fig 8: Mountain, glacier, buildings output: Grayscale / Colorized by Standard CNN / Real Image All respectively

5.2 U-Net Method A: Class-Average Histogram Fusion

5.2.1. Method A Training and Validation Loss Analysis For our second evaluation, we transitioned to the Histogram-Guided Fusion U-Net, utilizing **Method A**, where the model is conditioned on the average color palette of a specific class. Crucially, we shifted our objective function to Mean Absolute Error (MAE) to combat the averaging problem of MSE.

The training profile for this method demonstrates a more robust learning trajectory than the baseline. While the loss values (ranging between 0.025 and 0.075) are numerically higher than the MSE baseline due to the linear nature of MAE, the curve does not plateau prematurely. Instead, the validation loss tracks closely with the training loss, showing continuous improvement. Our Early Stopping mechanism identified **Epoch 59** as the point of optimal generalization, where the model effectively balanced feature extraction with color mapping without overfitting.

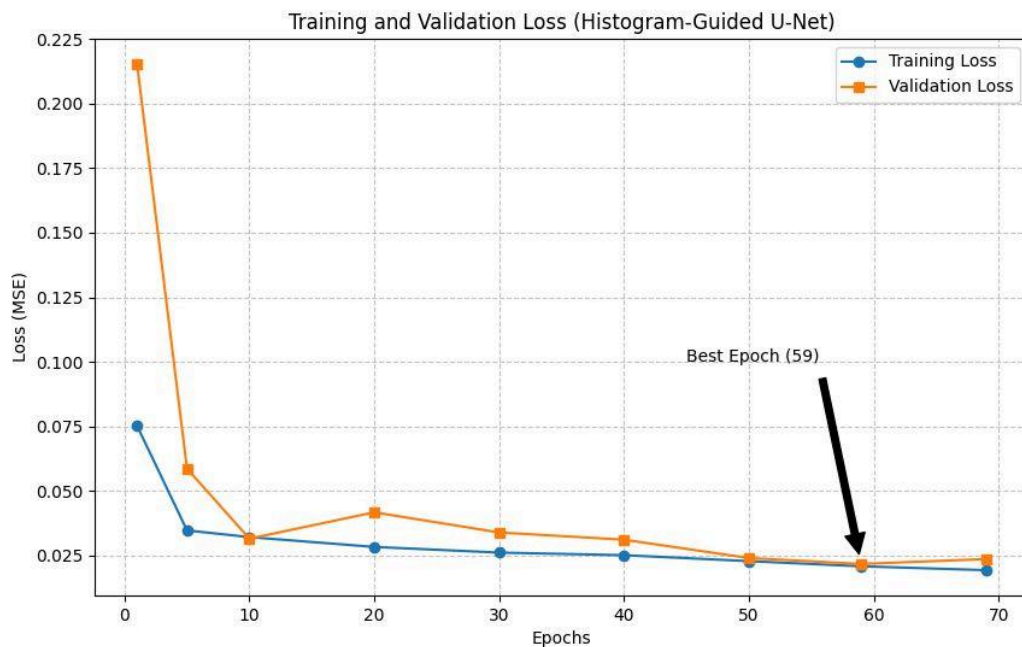


Fig 9: Training and Validation Loss Scores (MAE) of Standard UNet - 59 epoch

5.2.2. Method A Visual Evaluation: Class Consistency Visually, the application of Method

A yielded significantly more vibrant and semantically consistent results compared to the baseline. By calculating the average color palette for a whole class (e.g., "Forest") and providing it as a hint, the model successfully applied consistent greenery to forest scenes.

The comparison images demonstrate that the "Voltron" upgrades—specifically the Fusion Layer and MAE loss—successfully mitigated the desaturation problem. For example, when applying the average palette to "Street" and "Forest" images, the model produced outputs that were sharp (due to U-Net skip connections) and colorful, closely matching the tonal distribution of the ground truth rather than reverting to sepia tones.



Fig 10: Buildings output: Grayscale / Colorized by Standard U-net / Real image



Fig 11: Mountain output: Grayscale / Colorized by Standard U-net / Real image

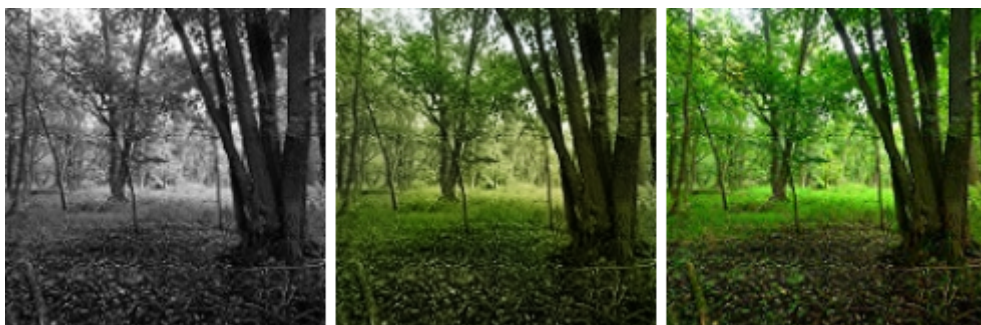


Fig 12: Forest output: Grayscale / Colorized by Standard U-net / Real image



Fig 13: Sea output: Grayscale / Colorized by Standard U-net / Real image



Fig 14: Glacier output: Grayscale / Colorized by Standard U-net / Real image



Fig 15: Forest output: Grayscale / Colorized by Standard U-net / Real image

5.3 U-Net Method B: Reference-Based Histogram Fusion

5.3.1. Method B Training and Validation Loss Analysis In the final and most complex phase, **Method B**, we evaluated the model's ability to perform Reference-Based Style Transfer, where specific (rather than average) histograms were used. This required the model to adapt dynamically to diverse histogram inputs for the same class.

The training profile for this run indicates a highly active optimization process. The best weights were restored at **Epoch 36**, which marked the point of lowest validation loss before the model began to potentially memorize specific histogram-image pairs. The convergence behavior here confirms that the fusion layer successfully learned to utilize the histogram as a dynamic variable, adjusting the output colors based on the varying input vectors provided

during training.

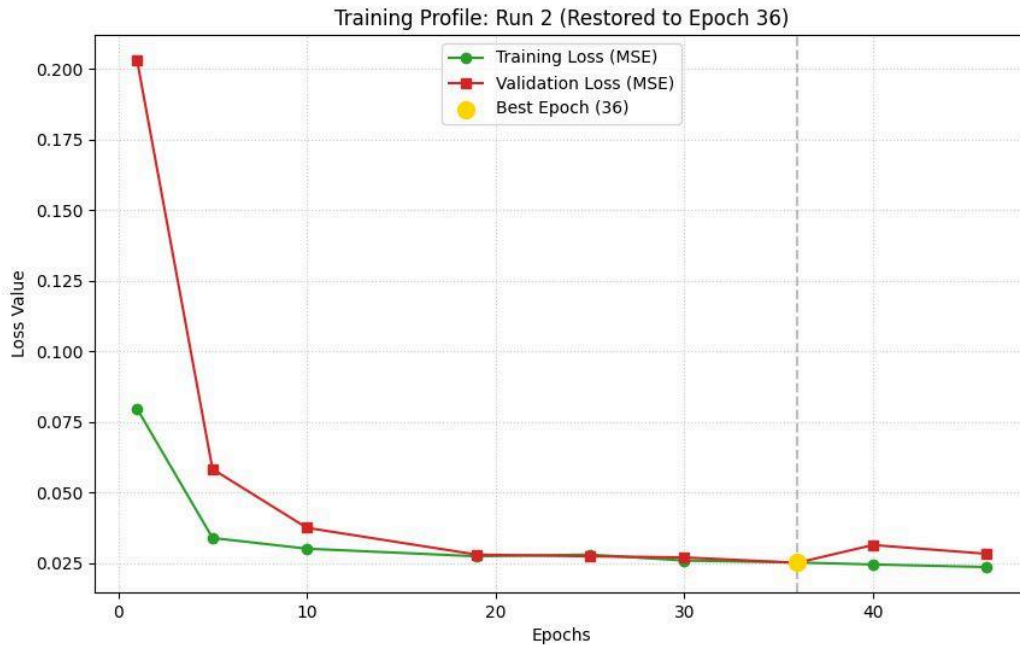


Fig 16: Training and Validation Loss Scores (MAE) of Reference Based UNet - 36 epoch

5.3.2. Method B Visual Evaluation: The Cross-Colorization Matrix The qualitative success of our project is best exemplified by the Cross-Colorization Matrix, which verifies the model's controllability. In this experiment, we applied six different class palettes to the same grayscale inputs to verify that the model was adhering to the histogram guidance rather than semantic memorization.

The visual proof is compelling. For instance, when a grayscale image of a mountain was conditioned with a "Buildings" histogram, the model adapted the mountain's colors to match the reddish-browns and grays of the reference building. Conversely, when the same mountain was conditioned with a "Glacier" histogram, the landscape was rendered in cold blues and whites.

This matrix demonstrates that our model has successfully disentangled content from style. It preserves the structural details of the input (e.g., the shape of the mountain) while faithfully hallucinating colors based on the provided histogram "hint," proving that the Histogram-Guided Fusion U-Net is capable of user-directed artistic colorization.



Fig 17: **Buildings output:** Grayscale / Building Style Colorization / Forest Style Colorization / Glacier Style Colorization / Mountain Style Colorization / Sea Style Colorization / Street Style Colorization / Real image



Fig 18: **Sea output:** Grayscale / Building Style Colorization / Forest Style Colorization / Glacier Style Colorization / Mountain Style Colorization / Sea Style Colorization / Street Style Colorization / Real image



Fig 19: **Forest output:** Grayscale / Building Style Colorization / Forest Style Colorization / Glacier Style Colorization / Mountain Style Colorization / Sea Style Colorization / Street Style Colorization / Real image

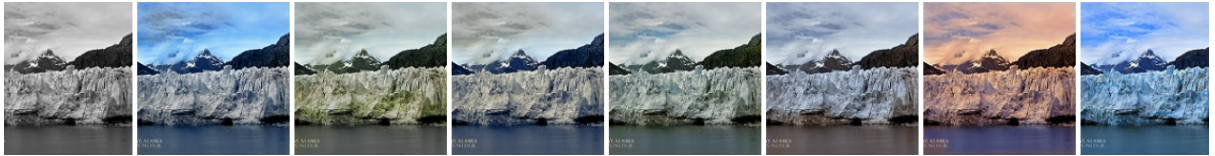


Fig 20: **Glacier output:** Grayscale / Building Style Colorization / Forest Style Colorization / Glacier Style Colorization / Mountain Style Colorization / Sea Style Colorization / Street Style Colorization / Real image

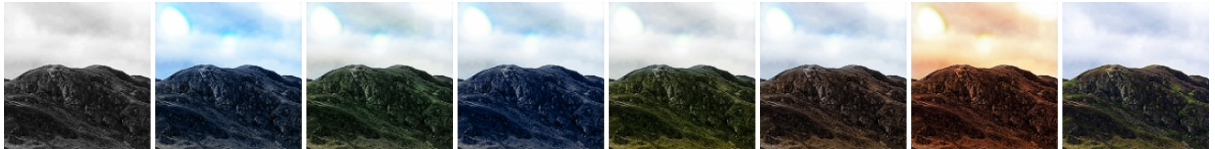


Fig 21: **Mountain output:** Grayscale / Building Style Colorization / Forest Style Colorization / Glacier Style Colorization / Mountain Style Colorization / Sea Style Colorization / Street Style Colorization / Real image



Fig 22: **Street output:** Grayscale / Building Style Colorization / Forest Style Colorization / Glacier Style Colorization / Mountain Style Colorization / Sea Style Colorization / Street Style Colorization / Real image

6. Conclusion

In this study, we addressed the inherent ill-posed nature of Single-Image Colorization, transforming what is typically treated as a regression task into a controllable, context-aware generative process. Through a systematic evaluation of three architectural phases, we successfully followed the complex trade-offs between numerical optimization and perceptual quality.

Our research found something about the numbers we use to measure how well computers can generate things. We did some tests. Found out something that did not make sense at first. The simple Baseline CNN, which was trained using Mean Squared Error did better on the numbers. Learned faster than our more complex models.. Even though the Baseline CNN was better with the numbers it was not as good at making things that looked real. The Baseline CNN and our advanced models were supposed to make things that looked real. The Baseline CNN did not do a good job of this even though it was good, with the numbers. By strictly minimizing the pixel-wise Euclidean distance, the CNN was penalized for any deviation from the ground truth, forcing it to predict the statistical average of all plausible colors—resulting in the monochromatic "Sepia Effect." This confirms that lower loss does not necessarily imply better colorization; rather, it often indicates a model that has learned to "play it safe" by averaging out multimodal ambiguities.

To overcome this limitation, we developed the Histogram-Guided Fusion U-Net. By shifting the objective function to Mean Absolute Error (MAE) and injecting explicit global color priors, we accepted a statistically higher variance in exchange for perceptually vibrant and semantically coherent results. Our results demonstrate that:

Structural Fidelity: The U-Net architecture is essential for recovering the high-frequency spatial details lost by standard autoencoders.

Controllability: The fusion mechanism successfully disentangles content from style, allowing users to manipulate the output mood via histogram injection.

Perceptual Quality: Optimization strategies like MAE loss and Saturation Boosting are necessary to align the model's objective with human visual perception, effectively breaking the regression-to-the-mean trap.

Ultimately, this project highlights that solving the colorization problem requires more than just deeper networks; it demands a fundamental rethinking of how we guide models through the uncertainty of the latent space. Our proposed Fusion architecture offers a robust framework for bridging the gap between automated feature extraction and artistic user control.